



SDU International Summer School

Introduction to R

Day 3 - Conditions, Loops, and Functions

Paola Vazquez Castillo

5 August 2026

Contents

1	Introduction	1
2	Conditions	2
2.1	Logical comparisons	2
2.2	Conditional statements	3
3	Functions	4
4	Loops	5
5	Functions (and conditions, and loops)	7
5.1	Using AI to write functions	8
6	Glossary of Functions	10
7	Exercises	10

1 Introduction

R Programming is a way to simplify analyzing larger amounts of data by using different properties of general programming into data analysis. Some of those properties are the *Conditions, Loops, and Functions*. These three elements are the basic tools that allow a code to 1) make decisions, 2) repeat work, and 3) become reusable. Conditions give the program a rule to behave differently depending on the data. Loops let you repeat the same operation over many values, files, countries, years, . . . instead of copying code. While functions let you abstract your code into a reusable tool, which makes scripts shorter and clearer.

2 Conditions

Conditions are statements that we give to a program to allow it to take decisions. That means it is a rule or logical statement that our code tests to decide where to continue next. Conditions are natural to all programming languages, as they are also in our daily language. For example:

- *If* my dog eats grass, he throws up.
- *If* my dog goes out for a walk, he is happy. *Else* he gets grumpy.

2.1 Logical comparisons

A comparison asks a yes/no question and returns TRUE or FALSE. Comparisons are the building block to do data analysis.

Symbols to define a comparison:

- == : equal to
- != : Not equal to
- > : Bigger than
- >= : Bigger than and equal to
- < : Smaller than
- <= : Smaller than and equal to

```
# Comparison operators in single values
```

```
170>185
```

```
## [1] FALSE
```

```
170==185
```

```
## [1] FALSE
```

```
170!=185
```

```
## [1] TRUE
```

We can also do this in real data. For example, we read the file `Heights_2019_slice.csv`. It is a file that contains information on the average height of people aged 19 in 2019 for 10 countries. Source: <https://www.ncdrisc.org/data-downloads-height.html>

```
# Tell R where to look for my files
```

```
setwd()
```

```
# Read our csv file
```

```
data_heights<-read.csv("Heights_2019_slice.csv")
```

```
# How does it look?
```

```
head(data_heights)
```

```
##      Country  Height
## 1  Australia 171.7200
## 2  Bangladesh 158.7295
## 3    Brazil 169.0702
## 4   Denmark 175.6816
## 5     Japan 165.2833
## 6    Mexico 164.0942
```

```
# How many rows and columns?
dim(data_heights)

## [1] 10 2

dk<-data_heights$Height[data_heights$Country=="Denmark"]
dk

## [1] 175.6816

# Was the average height higher than 165?
dk>=165

## [1] TRUE

# Was the height NOT higher than 175
!(dk > 175)

## [1] FALSE
```

We can also test for multiple conditions at the time. For example: - If my dog is happy *and* it is not raining, we don't go for a walk. - If my dog is grumpy *or* it is sunny, we go for a walk.

The first condition is *and*. That means that the first condition *and* the second one must be true for it to return TRUE. The second clause, is *or*, that means that as long as one of the clauses is TRUE, then the return value will be TRUE. In R, we use & for representing *and* and | for representing *or*.

```
# Testing two conditions
# & (and):
dk > 165 & dk < 175

## [1] FALSE

# | (or):
dk < 160 | dk > 165

## [1] TRUE
```

2.2 Conditional statements

As in normal language, it is also possible to return an output based on a condition with an *if* statement. `if()` only runs a block when a comparison is TRUE. If a condition is not true, the block stops unless we have specified what to do with an `else()`. The function `if(){}` is defined with:

- Inside the round brackets, write the condition(s).
- Inside the curly brackets (braces), write the function or expression to run if the condition is true.
- The `else{}` function can also be used to tell R what to do when the condition is not met.

#Example 1: Positive vs negative values

```
x<-10

if(x>0){
```

```
print("Positive value")
}
```

```
## [1] "Positive value"
```

```
#Add an else condition
```

```
x<- -5
```

```
if(x>0){
  print("Positive value")
}else{
  print("Negative value")
}
```

```
## [1] "Negative value"
```

```
#Example 2: Heights
```

```
mean_h<-mean(data_heights$Height)
```

```
if(dk>mean_h){
  print("Above average")
}else{
  print("Below average")
}
```

```
## [1] "Above average"
```

3 Functions

As seen in Day 1, R has built-in functions. But, in some cases, the operation you want to do is not available in R. It is then possible to write your own function.

To do so, you can use the function `function(){}:`

- Inside the round brackets, write the arguments of the function.
- Inside the curly brackets (braces), write the function or operation.
- Finally, you have to assign which output of the function with **return**.

```
# Function with one argument:
```

```
# cm to inches
```

```
cm_to_inch <- function(cm){
  inches <- cm / 2.54
  return(inches)
}
```

```
cm_to_inch(dk)
```

```
## [1] 69.166
```

```
cm_to_inch(180)
```

```
## [1] 70.86614
```

```
# Function with two necessary arguments
```

```
# Estimate BMI
```

```
bmi<-function(weight, height){
  value<-weight/(height^2)
  return(value)
}
```

```
bmi(weight = 90, height = 1.68)
```

```
## [1] 31.88776
```

```
bmi(weight = 65, height = 1.50)
```

```
## [1] 28.88889
```

```
# Function with three arguments (two necessary one optional)
```

```
# Estimate BMI
```

```
bmi_round<-function(weight, height, decimals=2){
  value<-round(weight/(height^2), digits = decimals)
  return(value)
}
```

```
# Specifying decimals
```

```
bmi_round(weight = 90, height = 1.68, decimals = 5)
```

```
## [1] 31.88776
```

```
# Using the default decimals
```

```
bmi_round(weight = 65, height = 1.50)
```

```
## [1] 28.89
```

4 Loops

In programming, loops are used to repeat a specific code or block of codes. In R, the for-loop is often used. The function means that: *for* specific values or positions, *loop* the selected function.

To do a loop, you can use the function `for(){}`:

- Inside the round brackets, write the sequences, values or positions to take on each loop.
- Inside the curly brackets (braces), write the function or operation to be repeated.
- Print the results or save them in a new object

```
# For-loop using values
```

```
#Define specific values
```

```
x<-c(1,2,4,7,10)
```

```
#Add 10 to each values in x
for(values in x){
  output=values + 10
  print(output)
}
```

```
## [1] 11
## [1] 12
## [1] 14
## [1] 17
## [1] 20
```

```
# for-loop using position
```

```
#Add 10 to values in x in the 2nd and 4th position
for(position in c(2,4)){
  output=x[position] + 10
  print(output)
}
```

```
## [1] 12
## [1] 17
```

We can also use both conditions and loops at the same time. This is very frequent to do for example when creating a new variable. Let's test that with our real data:

```
# We create a vector with the countries in our dataset
countries<-data_heights$Country
```

```
# Loop over values:
```

```
# here I am defining when does this needs to be repeated
```

```
for(country in countries){
  # here we "filter" h as a value to be replaced each iteration (changing country each time)
  h <- data_heights$Height[data_heights$Country == country]
  # condition
  if(h > mean_h){
    print(paste(country, "is above average"))
  } else {
    print(paste(country, "is below average"))
  }
}
```

```
## [1] "Australia is above average"
## [1] "Bangladesh is below average"
## [1] "Brazil is above average"
## [1] "Denmark is above average"
## [1] "Japan is below average"
## [1] "Mexico is below average"
## [1] "Nepal is below average"
## [1] "Poland is above average"
## [1] "Spain is above average"
## [1] "United States of America is above average"
```

```
# Loop over positions:

for(i in 1:length(countries)){
  h <- data_heights$Height[i]
  # condition
  if(h > mean_h){
    print(paste(countries[i], "is above average"))
  } else {
    print(paste(countries[i], "is below average"))
  }
}
```

```
## [1] "Australia is above average"
## [1] "Bangladesh is below average"
## [1] "Brazil is above average"
## [1] "Denmark is above average"
## [1] "Japan is below average"
## [1] "Mexico is below average"
## [1] "Nepal is below average"
## [1] "Poland is above average"
## [1] "Spain is above average"
## [1] "United States of America is above average"
```

5 Functions (and conditions, and loops)

As we saw functions are useful when we have to reuse an operation in multiple values. We can for example want to have a function for classifying the height of different countries or different reference cuts.

```
classify_country <- function(data, country, reference = mean_h){
  h <- data$Height[data$Country == country]
  if(h > reference){
    return("above average")
  } else {
    return("below average")
  }
}
```

```
# Testing the function
classify_country(data_heights, "Denmark")
```

```
## [1] "above average"
```

```
# Changing the reference
classify_country(data_heights, "Mexico", reference=150)
```

```
## [1] "above average"
```

We could also want to have a function that combines a previously defined function with a loop:

```

classify_all <- function(data){
  # We are creating a data frame with two columns
  results <- data.frame(Country = data$Country,
                        Group = NA)
  # We are running a loop by position
  for(i in 1:length(data$Country)){
    # We are reusing our previous function
    results$Group[i] <- classify_country(data,data$Country[i])
  }
  return(results)
}

classify_all(data_heights)

```

```

##           Country      Group
## 1      Australia above average
## 2    Bangladesh below average
## 3        Brazil above average
## 4      Denmark above average
## 5         Japan below average
## 6        Mexico below average
## 7         Nepal below average
## 8        Poland above average
## 9         Spain above average
## 10 United States of America above average

```

5.1 Using AI to write functions

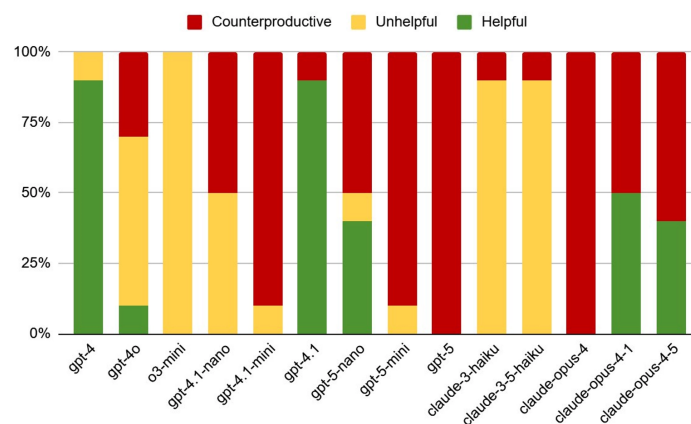
Language models can write functions and loops, but it becomes very important to understand what we want to do and what is the code provided by AI doing. A good workflow is: (1) state the goal or formula precisely in your prompt, (2) read the generated code line by line, and (3) test it on an input where you already know the answer.

The example below was produced by an AI assistant to create a new column equal to `index_value + 1` in our `data_heights` data frame.

```
data_heights$new_column <- data_heights$index_value + 1
```

```
## Error in $<-.data.frame (*tmp*, new_column, value = numeric(0)) :replacement
has 0 rows, data has 10
```

The code is valid R, but it fails because the AI invented or assumed a column called `index_value` that is not in the data. When asking AI for help to find out what is causing the error, this is how helpful different models were:



Source: Jamie Twiss (<https://spectrum.ieee.org/ai-coding-degrades>)

Another example below was produced by an AI assistant when we asked it to sort the data from the shortest to the tallest country.

```
# "sort my data from shortest to tallest"
data_sorted <- data.frame(
  Country = data_heights$Country,
  Height = sort(data_heights$Height)
)

head(data_sorted)
```

```
##      Country  Height
## 1  Australia 158.3764
## 2  Bangladesh 158.7295
## 3    Brazil 164.0942
## 4   Denmark 165.2833
## 5     Japan 169.0670
## 6     Mexico 169.0702
```

This runs without any error, and at a glance it looks right: the heights climb from smallest to largest. But it is wrong. Only the `Height` column was sorted, while `Country` stayed in its original order, so every row now pairs a country with the wrong height.

We can catch this by checking a value we already know. Earlier in this session Denmark's height was 175.68, so let us look it up again in the "sorted" data:

```
data_sorted$Height[data_sorted$Country == "Denmark"]

## [1] 165.2833
```

The value is no longer 175.68, because Denmark's row now holds another country's height. To sort correctly we have to reorder the whole rows together, not one column on its own, using `order()`:

```
data_sorted <- data_heights[order(data_heights$Height), ]
head(data_sorted)
```

```
##      Country  Height
## 7     Nepal 158.3764
```

```
## 2 Bangladesh 158.7295
## 6 Mexico 164.0942
## 5 Japan 165.2833
## 9 Spain 169.0670
## 3 Brazil 169.0702
```

This is the important lesson: “it runs” and “it is correct” are two different things. Code that returns a clean, plausible-looking result can still be wrong, and the only way to know is to check it against something you already know. AI makes a lot of mistakes, it is your responsibility to make sure it is doing what you need to do. That is why you need to understand how to write your own code. A good rule of thumb is using AI code only if you know what each line of the code is doing.

6 Glossary of Functions

Syntax	What it does	Example
<code>== != > >= < <=</code>	Compare two values, returns TRUE or FALSE	<code>170 > 165</code>
<code>& !</code>	Combine or negate conditions (and, or, not)	<code>dk > 165 & dk < 175</code>
<code>if(cond){ }</code>	Run the block only when the condition is TRUE	<code>if(x > 0){ print("positive") }</code>
<code>else{ }</code>	Run when all previous conditions are FALSE	<code>else{ print("negative") }</code>
<code>function(args){ return(x) }</code>	Define your own function; arguments in (), operation in {}, <code>return()</code> sets the output	<code>f <- function(cm){ return(cm/2.54) }</code>
<code>return()</code>	Send a value back as the function's output	<code>return(value)</code>
<code>for(i in x){ }</code>	Repeat the block once for each element or position of x	<code>for(country in countries){ }</code>
<code>print()</code>	Display a result (needed to see output inside a loop)	<code>print(output)</code>
<code>paste()</code>	Join text and values into one string	<code>paste(country, "is above average")</code>

7 Exercises

For these exercises, use the `data_heights` object from this session.

- Pick one country from the dataset. Extract its height, then use an `if else` statement to print whether it is "tall", "average" or "short". Use 172 cm and 165 cm as your thresholds (above 172 is tall, below 165 is short, anything in between is average).
- Turn that logic into a function called `height_category()` that takes a height as its argument and returns "tall", "average" or "short". Test it on three different heights. Hint: you need multiple `if` (the same structure as above, wrapped in a function).
- Using a `for` loop, apply your function to every country in `data_heights` and print each country next to its category.

- Hint: reuse `height_category()` inside the loop, and use `paste()` to combine the country name with its category.
- Without using the `mean()` function, use a for loop to add up all the heights and divide by the number of countries. Check that your result matches `mean(data_heights$Height)`. (Hint: start with a total of 0 before the loop, then add each height to it one at a time).
- (Extra) Write a function that takes `data_heights` and returns the data with an extra column giving each country's difference from the mean height (its height minus the average of all heights). Which country is furthest from the average?